Swinburne University of Technology

COS30049

Computing Technology Innovation Project

# Assignment 3

**Full-Stack Web Development for AI Applications in Cybersecurity Scenario**

**Author :** Truong Ngoc Gia Hieu
Duong Nguyen Dang
Pham Dinh Duc Anh
Pahan Pathirathna

**Lecturer**: Mr. Hoang (Author) Nguyen

**Due Date**: 12th April, 2026

**Submission Date:** 12th April, 2026

**Word Count:** 3444 words

# Table of Contents

**Part A: Abstraction** ..... 1
- **1.1 Assignment Requirements** ..... 3
- **1.2 Project Overview** ..... 3
- **1.3 Scope of the Project** ..... 4
- **1.4 Executive Summary** ..... 4

**Part B: Backend Integration** ..... 5
- **2.1 Overall Architecture and Framework Selection** ..... 5
- **2.2 Model Serialization and Loading** ..... 5
- **2.3 API Endpoint Design** ..... 6
- **2.4 Data Processing Workflow** ..... 7
- **2.5 Error Handling and Validation** ..... 7

**Part C: Frontend Integration** ..... 8
- **3.1 Architecture Diagram** ..... 8
- **3.2 Frontend Components and User Interface** ..... 9
- **3.3 Prediction** ..... 10

**Part D: Future Enhancements** ..... 11
- **4.1 Advanced Model Evolution** ..... 11
- **4.2 Scalable Data Infrastructure** ..... 12
- **4.3 Explainable AI (XAI) for User Trust** ..... 12
- **4.4 Multi-Modal Expansion and Edge Deployment** ..... 13
- **4.5 AWS Architecture Migration** ..... 13

**Part E: Conclusion** ..... 14

# Part A: Abstraction

## 1.1 Assignment Requirements

In the third assignment of COS30049, the team is tasked to design and implement a complete full-stack web application integrated with machine learning models to solve a real-world problem. Furthermore, the project focuses on applying advanced algorithms, system design principles, and software development techniques to build a functional system for end-users, ensuring that all components are linked together seamlessly and professionally justified**.**

## 1.2 Project Overview

During participating in the COS30049 unit, our team has developed an integrated security platform that utilizes machine learning models to detect and mitigate digital threats. In detail, the system serves as a 'smart filter' capable of analyzing three major categories: Fake News (URLs), Spam & Ham Emails, and Malware Detection. By processing user-provided inputs, the application delivers real-time risk assessments and probability distributions, transforming complex AI predictions into actionable insights for end-users.

Furthermore, the primary goal of this project is to deliver a fully functional web ecosystem that provides high-accuracy threat detection. Therefore, we expect the innovative platform to not only identify malicious content but also to visualize data through interactive metrics such as confidence scores and probability charts. Ultimately, the project will result in a professional-grade platform supported by comprehensive technical documentation and a scalable system design

Next, the journey throughout the COS30049 unit demonstrates a significant progression in technical complexity and system design:

- Assignment 1 (Project Proposal and Planning): In the initial phase, our team focused on defining the project scope and identifying a real-world problem. Besides, we established a project roadmap, conducted feasibility studies for our AI solutions, and designed the preliminary system architecture to ensure a solid foundation.
- Assignment 2 (Data Processing and Model Evaluation): In the second assignment, we shifted to the core intelligence of the platform and performed extensive data cleaning and preprocessing on real-world datasets and conducted rigorous model evaluations. By analyzing metrics such as accuracy, precision, and recall, we selected the most effective Machine Learning models for threat detection.
- Assignment 3 (Full-stack Integration and Implementation): In this final phase, we are architecting a complete full-stack application. Additionally, the primary goal is to link together our evaluated AI models with a robust FastAPI backend and an interactive web frontend, transforming our analytical research into a functional security platform for end-users.

## 1.3 Scope of the Project

Then, the scope of this fascinating project concentrates on developing a unified security ecosystem, structured into three primary components:

- Core Intelligence: In the application, we implement three specialized Machine Learning models: Multinomial Naive Bayes for Spam & Ham detection, and Random Forest for Fake News (URLs) and Malware classification. These models are designed to deliver high-accuracy probability distributions, providing users with precise risk assessments for each digital threat.
- System Integration: The project involves building a robust full-stack architecture that integrates a FastAPI backend for high-performance model inference with a modern web frontend developed using Vue.js (and Tailwind CSS for styling). As a result, this seamless connection ensures real-time communication between the AI engines and the user interface, providing a responsive and intuitive experience
- Functional Deliverables: The platform allows users to input raw data (URLs or text) and receive real-time risk assessments, visualized through intuitive metrics and confidence scores.1.4 Executive Summary

## 1.4 Executive Summary

TruthGuard is an advanced AI-driven platform designed to combat the proliferation of digital misinformation through real-time content verification. This project integrates a robust Machine Learning pipeline with a modern Vue.js frontend to provide users with instantaneous, data-backed risk assessments:

- Hybrid Detection Engine: Utilizing optimized algorithms to achieve high predictive accuracy in identifying malicious linguistic patterns.
- Scalable Infrastructure: Designed for future migration to AWS cloud-native services, ensuring enterprise-grade reliability and low-latency performance.
- Transparency & Ethics: A strategic roadmap focused on Explainable AI (XAI) to demystify model decision-making and build user trust.

Ultimately, TruthGuard serves as a critical defense mechanism in the modern information landscape, delivering a seamless bridge between complex AI analysis and intuitive user protection

# Part B: Backend Integration

## 2.1 Overall Architecture and Framework Selection

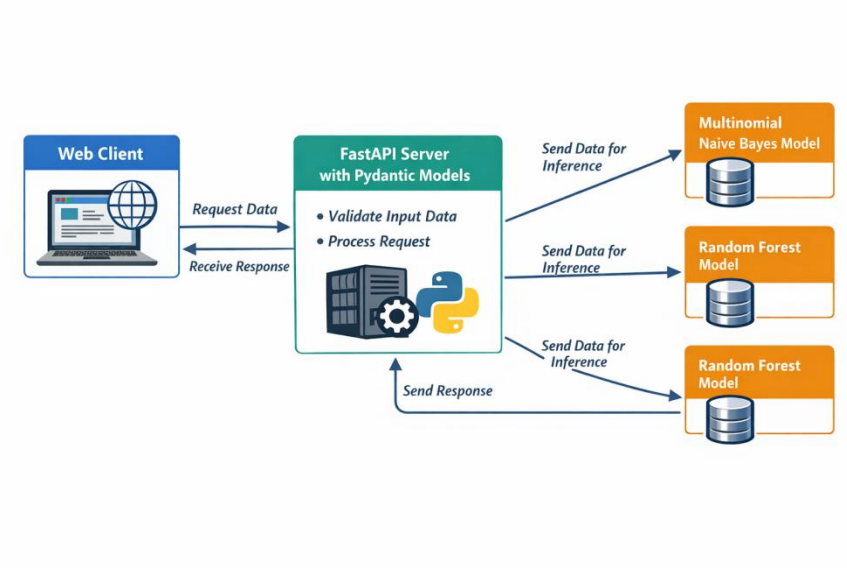


***Figure 1: Logical Data Flow and Backend Integration Architecture.***

It is clear that the main focus of the architecture is on the smooth flow of information between the client side and the backend intelligence layer. Furthermore, the process is initiated when the Web Client (developed with Vue.js) sends a request to the FastAPI Server. Subsequently, the backend utilizes Pydantic models for rigorous input validation, ensuring data integrity before the data is routed to specialized machine learning engines including Multinomial Naive Bayes and Random Forest for inference. Particularly, the integration process follows a structured pipeline:

- Data Validation & Processing: Upon receiving a request from the Web Client, the FastAPI server utilizes Pydantic to ensure input integrity before performing real-time preprocessing (vectorization).
- Model Inference: The system then routes the processed data to the specific specialized engine—Multinomial Naive Bayes for Spam detection or Random Forest for Fake News and Malware analysis.
- Inference Response: Each model performs a probabilistic prediction, and the result is encapsulated into a JSON response, which is then sent back to the client for immediate visualization."

## 2.2 Model Serialization and Loading

In the model serialization and loading part, the team ensures that the intelligence engines are portable and ready for production. This process is executed through three concise steps:

- Serialization with Joblib: Our team utilize the Joblib library to serialize the trained Multinomial Naive Bayes and Random Forest models into

binary `.pkl` files. As a result, this approach allows the system to store the exact weights and decision logic obtained from the training phase.

- Vectorizer Preservation: Alongside the models, the TfidfVectorizer is also serialized which is crucial to ensure that the numerical mapping of input data remains consistent between the training and inference stages, preventing feature mismatch.
- Startup Loading Mechanism: For minimizing latency, all `.pkl` artifacts are loaded into the system memory during the FastAPI startup event. By residing in RAM, the models can provide near-instantaneous predictions for every user request without repetitive I/O operations

## 2.3 API Endpoint Design

Regarding of facilitating communication between the Vue.js frontend and the AI models, my team designed a centralized RESTful endpoint using FastAPI. As a result, we believe that design ensures that all user inputs whether URLs, emails, or code snippets are processed through a single, secure gateway.

```python
@app.post("/analyze")
async def analyze(request: ScanRequest):
    """
    Main Endpoint for Frontend.
    Returns all vital metrics for Figma UI components.
    """
    try:
        category, ai_results = input_router.predict_logic(request.content, handler)

        # Wrapping everything for a professional Frontend Showcase
        return {
            "meta": {
                "scan_id": id(request.content),
                "timestamp": datetime.now().isoformat(),
                "category_detected": category
            },
            "ui_metrics": ai_results
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

***Figure 2: FastAPI Endpoint Implementation for Centralized Request Routing and Pydantic Data Validation.***

As indicated in ***Figure 2***, it displays the actual implementation of the /analyze endpoint within the FastAPI framework. Additionally, it showcases the use of a ScanRequest class inherited from Pydantic to enforce strict data types for incoming user content. Besides, the code also shows a modular approach, with the input_router acting as a layer that makes decisions by choosing the right machine learning handler based on the input's characteristics. Finally, the function encapsulates the results into a standardized JSON response, providing organized metadata and UI-ready metrics for the frontend.

## 2.4 Data Processing Workflow

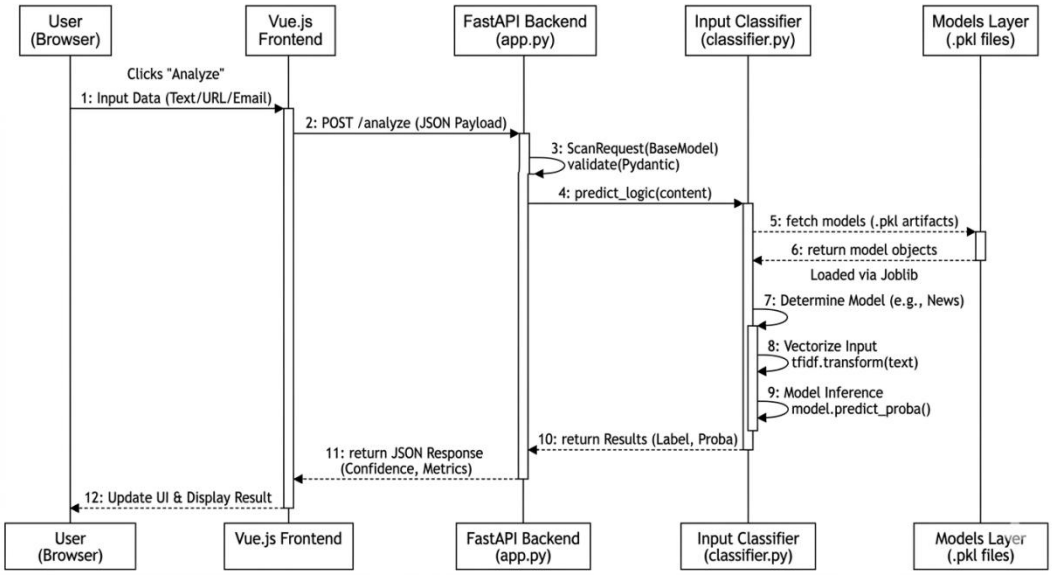


***Figure 3: Sequence Diagram of the Integrated Data Processing Workflow***

Subsequently, ***Figure 3*** highlights integrated data processing workflow, showcasing the sequential interactions between the system's components over a single request. Specifically, the diagram demonstrates a highly structured and stateless communication pattern including:

- Step 1 (Transmission): The Vue.js Frontend sends a POST request with a JSON payload to the /analyze endpoint in ***app.py***.
- Step 2 (Validation): The FastAPI Backend triggers Pydantic (ScanRequest) to validate the input structure, ensuring data integrity before processing.
- Step 3 (Routing): The Input Classifier (***classifier.py***) analyzes the raw string (Regex for URLs, Keywords for Malware) to dynamically route the request to the correct AI logic.
- Step 4 (Inference): The Models Handler performs text vectorization and executes the prediction using pre-loaded .pkl models (predict_proba).
- Step 5 (Response): The system returns a comprehensive JSON response (Labels, Confidence, Risk Scores) to the Frontend for immediate UI update.

## 2.5 Error Handling and Validation

Finally, this section outlines the mechanisms ensuring system stability and data integrity during the integration process:

- Schema Validation: The backend utilizes Pydantic (ScanRequest) to enforce strict data types. Any request lacking the "content" field is

automatically rejected with a 422 Unprocessable Entity error before reaching the AI logic.

- Input Sanitization: Within classifier.py, the predict_logic function applies .strip() and Regex matching to handle empty strings or malformed URLs, ensuring that only valid data is routed to the inference engines.
- Global Exception Handling: The main /analyze route is wrapped in a Try-Except block. This catches runtime anomalies (e.g., model loading issues) and returns a structured 500 Internal Server Error, preventing backend crashes while providing clear feedback.

In summary, the integration of the backend infrastructure successfully transitions theoretical AI models into a functional web service. Therefore, we believe that the system delivers a responsive and reliable security analysis tool by combining FastAPI's high-performance routing with Joblib's efficient serialization,. Notably, the implementation of a centralized, validated API ensures that the platform is production-ready, providing real-time intelligence with optimized latency for the final user interface.

# Part C: Frontend Integration

## 3.1 Architecture Diagram

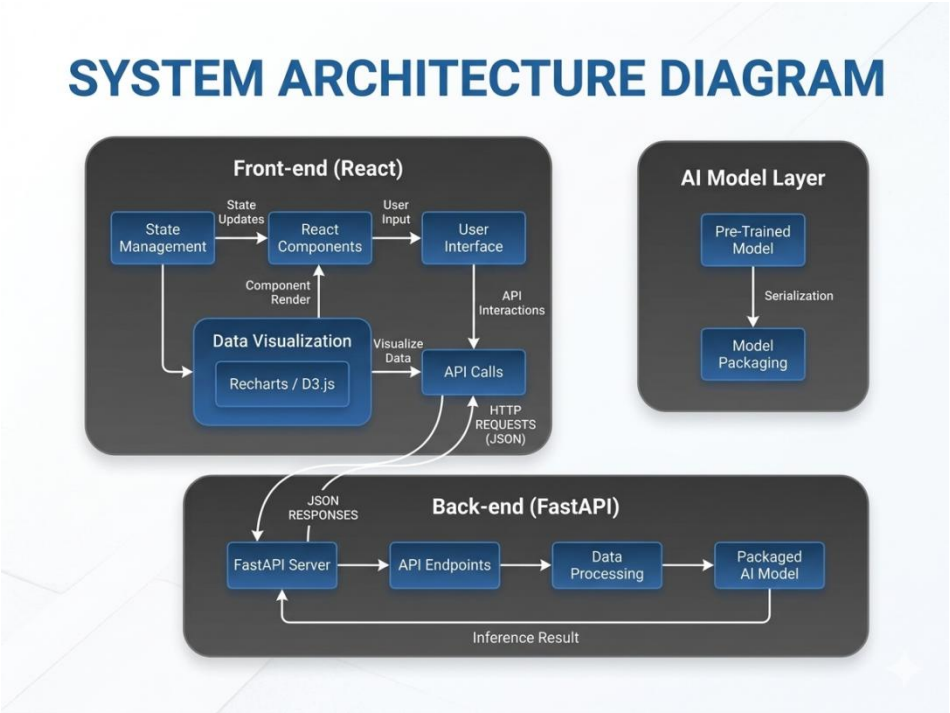


***Figure 4: Unified Three-Tier System Architecture for AI-Driven Security Analysis***

As illustrated in ***Figure 4,*** the system follows a decoupled three-tier architecture, ensuring high modularity and seamless data flow between the user interface and the machine learning engines:

- Front-end Tier (React/Vue.js): First, this laye acts as the presentation and state management layer and captures user inputs and utilizes

Asynchronous API Calls to communicate with the backend. Specifically, it is responsible for rendering real-time Data Visualizations (via Recharts/D3.js) based on the intelligence received.

- Back-end Tier (FastAPI): Secondly, the back-end tier serves as the orchestration hub. Additionally, the FastAPI Server handles request routing through specific API Endpoints and performs essential Data Processing including validation and sanitization before interfacing with the models.
- AI Model Tier: Represents the core intelligence of the system. It consists of Pre-trained Models that have been Packaged (serialized into .pkl files). These models receive processed data, execute probabilistic inference, and return structured results to the backend for final delivery.

## 3.2 Frontend Components and User Interface

In this part, our team highlights the modular components developed in Vue.js that transform raw AI predictions into an intuitive security dashboard.

At the core of the user experience is the Input Hub, a centralized interface designed to capture multi-format content such as URLs, emails, or text snippets. Therefore, this component utilizes Reactive State Management to monitor user input and trigger asynchronous API calls via Axios upon interaction. By implementing real-time input sanitization, the hub ensures that only valid data strings are transmitted to the backend, maintaining system stability from the first point of contact.

Once the backend processes the request, the Result Visualization Engine dynamically renders the data using Recharts/D3.js. Particularly, this module maps the *ui_metrics.confidence_score* and *risk_score* onto a high-fidelity Risk Meter, providing immediate clarity through color-coded indicators such as Red for high-risk threats. Consequently, my team believes that these gauges are interactive distribution charts that break down the AI's probabilistic reasoning, effectively translating "black box" machine learning logic into transparent, visual analytics.

Finally, for bridging the gap between raw data and user safety, the interface concludes with a Proactive Recommendation Module. As a result, this component interprets the prediction results to display tailored security advice, such as "Content is verified" or "High risk: Do not click". By converting numerical probability into actionable insights, this module empowers users to make informed decisions, ensuring the platform serves as a practical tool for real-time security assessment.

## 3.3 Prediction

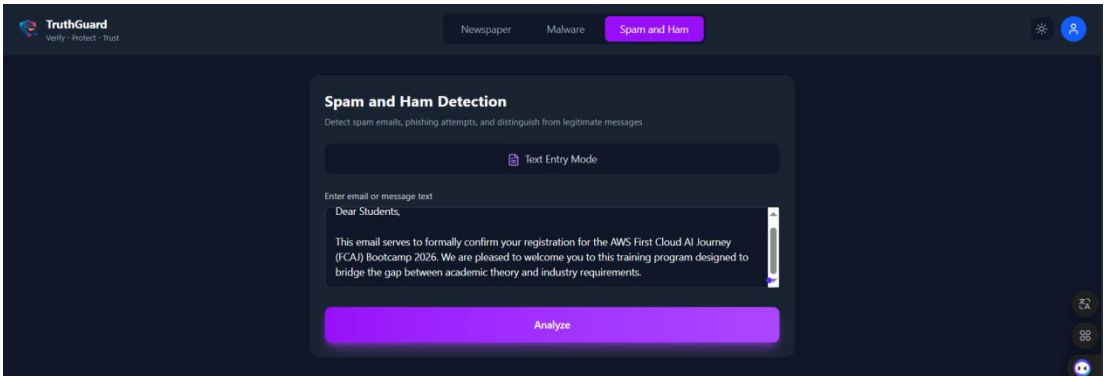


*Figure 5: TruthGuard Unified Security Dashboard*

Next, it is clear that the primary user interface of TruthGuard which is designed as a Single Page Application (SPA). Furthermore, the interface features a clean, dark-themed Intelligence Hub engineered to unify the platform's diverse security services. In addition, users can seamlessly switch between specialized detection modes including Newspaper (Fake News), Malware, and Spam and Ham via the top navigation bar.

Besdies, the active module, Spam and Ham Detection provides a dedicated Text Entry Mode area. Subsequently, this component is designed to accept multi-line textual content, such as email body text such as Dear Students and so on for testing. Upon clicking the state-driven Analyze button, the frontend initiates an asynchronous API call to the FastAPI backend, triggering the specific NLP model to assess the text's legitimacy in real-time.

*Figure 6: Real-time Detection Confidence and Multi-dimensional Risk Analysis Dashboard*

As my team expected, the result showcases the Result Visualization Layer of TruthGuard which translates raw AI inference into intuitive visual metrics. The dashboard is divided into two primary analytical sections:

- Detection Confidence and Risk Scoring: The left panel presents a high-level summary, featuring a prominent Confidence Percentage (50.05%) and a categorical threat level indicator (Medium Threat). This is supported by a comparative bar chart that maps the relationship between Prediction Confidence and the Risk Score, allowing users to immediately gauge the certainty of the AI's assessment.

- Multi-dimensional Analysis: The right panel utilizes a Radar (Spider) Chart to provide a granular breakdown of the analysis. By evaluating five key dimensions including Confidence, Risk, Safety, Prediction Strength, and Reliability, this component offers a holistic view of the content's security profile. As a result, this visualization effectively deconstructs the machine learning model's logic, providing transparency and helping users understand the specific factors contributing to the final risk classification.

***Figure 7: Threat Activity Timeline and Categorical Classification Module***

Finally, the ***Figure 7*** displays the historical and predictive analytical components of the TruthGuard interface. In detail, the visualization is divided into two functional areas:

- Threat Activity Timeline: A line chart that tracks threat intensity over a specified duration (from the current moment to +2h). This component provides users with a temporal perspective on security trends, allowing for proactive monitoring of potential risk escalations over time.
- Detected Threats Summary: The final classification output, which in this instance identifies the content as 'Ham' with a 'Low' risk tag. This module provides a clear, simplified verdict: *This content appears safe to use'* translating complex AI probability into a definitive security status for the end-user.

Finally, the frontend integration successfully converts complex AI data into a user-centric security dashboard. By leveraging Vue.js's reactive architecture and dynamic visualization tools, the system provides a transparent, real-time view of threat analysis. Conseuqently, this synergy between the Input Intelligence Hub and the Result Visualization Layer ensures that high-fidelity security insights are not only accurate but also actionable, bridging the gap between advanced data science and practical user safety.

# Part D: Future Enhancements

## 4.1 Advanced Model Evolution

First and foremost, the current platform faces inherent limitations in capturing complex linguistic structures and contextual nuances. By addressing these challenges, the next iteration will transition toward Transformer-based Deep Learning architectures to significantly enhance detection precision and semantic understanding.

- Implementation of Transformer Models: Future development aims to integrate models such as BERT or RoBERTa. By utilizing Attention Mechanisms, these models can analyze the context of words bidirectionally which allow for a superior understanding of intent compared to traditional feature-based classifiers.
- Semantic Nuance Detection: Leveraging pre-trained language models allows TruthGuard to identify "Deep-fake" text and sophisticated phishing attempts. As a result, my team believe that this transition focuses on capturing long-range dependencies and semantic subtleties, ensuring the system can differentiate between legitimate content and highly-curated misinformation.
- Dynamic Learning Pipelines: We propose a Continuous Learning Loop where the model is periodically fine-tuned on emerging threat datasets, ensuring the platform remains resilient against evolving tactics in digital deception, maintaining a high baseline of predictive reliability.

## 4.2 Scalable Data Infrastructure

Next, for managing expanding datasets and accelerate detection, the architecture will integrate a Vector Database such as Pinecone. Consequently, this transition shifts the system from repetitive inference to a high-performance, memory-enabled framework.

- Semantic Similarity Search: By storing high-dimensional embeddings of known threats, the system can identify similar malicious patterns instantaneously without re-running full model inference.
- Latency Optimization: Vector indexing enables sub-millisecond lookups across millions of records, drastically reducing computational overhead and ensuring real-time responsiveness.
- Self-Evolving Intelligence: This infrastructure facilitates a continuous feedback loop where new confirmed threats are indexed immediately, creating a resilient and up-to-date repository of global threat intelligence.

## 4.3 Explainable AI (XAI) for User Trust

Then, our website faces limitations in capturing complex linguistic structures. By addressing these challenges, the next phase of development will focus on integrating Explainable AI (XAI) frameworks to demystify the model's decision-making process and provide verifiable security insights

- Feature Importance Heatmaps: Integration of frameworks like SHAP or LIME will enable the UI to highlight specific linguistic patterns that triggered a high-risk score, ensuring transparency over arbitrary verdicts.
- User Education: Providing local explanations for individual predictions transforms TruthGuard into an educational platform, reducing user skepticism through logical evidence.
- Bias Mitigation: XAI tools allow for continuous auditing of the model's internal weighting, ensuring the system remains objective and free from unintended linguistic biases.

## 4.4 Multi-Model Expansion and Edge Deployment

Finally, our team aims to expand beyond text analysis toward multi-modal detection and localized edge processing to provide a comprehensive security suite. Consequently, this strategic shift ensures TruthGuard remains resilient against diverse threat vectors while maintaining near-instantaneous responsiveness.

- Multi-Modal Detection: Future iterations will incorporate the ability to analyze images and metadata within suspicious links. By integrating Computer Vision (CV) models, TruthGuard can identify visual phishing cues or malicious QR codes, offering a unified defense against diverse threat vectors.
- Edge-Side Processing: By leveraging TensorFlow.js or ONNX Runtime, lightweight versions of the detection models can be deployed directly to the user's browser. This "Edge Deployment" reduces dependency on constant backend communication, ensuring near-instantaneous analysis while enhancing user data privacy.
- Browser Extension Integration: To transition from a standalone web app to a proactive security companion, a dedicated browser extension will be developed. This will allow TruthGuard to scan active tabs and incoming emails in real-time, providing an extra layer of "always-on" protection for the user's digital environment.

## 4.5 AWS Architecture Migration

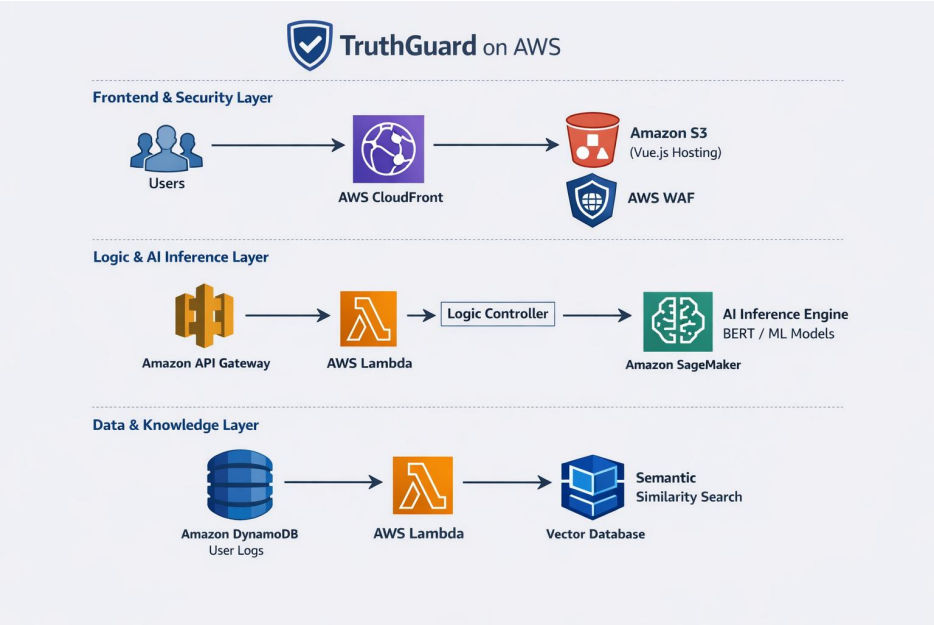


*Figure 8: Proposed Cloud-Native Architecture for TruthGuard on AWS*

Finally, for transforming TruthGuard into a production-ready enterprise solution, the architecture will transition to a fully managed Amazon Web Services (AWS) ecosystem. This strategic migration ensures high availability, automated scaling, and global deployment through a three-tier serverless framework.

- Frontend & Security Layer: The Vue.js application will be hosted on Amazon S3 and distributed globally via Amazon CloudFront. This ensures sub-millisecond latency for users worldwide, while AWS WAF provides a robust firewall against common web vulnerabilities.
- Logic & AI Inference Layer: By leveraging AWS Lambda and Amazon API Gateway, the system achieves a serverless "Logic Controller" environment. This connects directly to Amazon SageMaker, which serves as the core inference engine for BERT and other complex machine learning models, scaling compute resources dynamically based on traffic.
- Data & Knowledge Layer: The infrastructure utilizes a dual-database approach. Amazon DynamoDB stores structured user logs and threat history, while a specialized Vector Database enables "Semantic Similarity Search" to instantaneously identify recurring malicious patterns without re-running full model inference.

In conclusion, these enhancements transition TruthGuard from a functional prototype to an enterprise-grade ecosystem. By integrating Transformer-based models with a scalable AWS architecture, the platform ensures high-performance detection and global reliability. Furthermore, the adoption of Explainable AI and multi-modal capabilities provides a transparent, adaptive, and comprehensive defense mechanism, solidifying TruthGuard's resilience against the evolving complexities of digital misinformation

# Part E: Conclusion

In conclusion, this project has successfully developed TruthGuard, an AI-driven platform dedicated to enhancing digital integrity through automated misinformation detection. By leveraging a combination of Machine Learning models and a modern web framework, the system effectively addresses the critical need for real-time content verification. The research and implementation phases have demonstrated that while current technologies provide a solid foundation, the integration of Transformer-based architectures and cloud-native scalability is essential for long-term resilience. Ultimately, TruthGuard serves as a proactive defense mechanism, empowering users with the tools necessary to navigate the complexities of the modern information landscape with greater transparency and security.